

# Startup Matchmaking: Automation Approach

Task 2 write-up — architecture only, no build

Today, matching an incoming request to suitable startups means someone opening a 600+ row Excel file and eyeballing it against whatever the requester asked for — about 7 hours a week across 10+ requests. The fix is not a smarter spreadsheet; it's turning “read the email, search the sheet, write a reply” into a short pipeline with one required human checkpoint before anything goes out. Since we already run and fully control a server for the startin.lv website, this is designed to live there as an extension of that app rather than as a new stack of monthly subscriptions.

## Architecture and data flow

|                              |                                                                                                                                                                                                                              |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Intake</b>             | Request arrives at the existing team mailbox. A small polling script, running on the same server, checks it every few minutes and hands new messages to the app — no separate inbound-email service to pay for.              |
| <b>2. Understand the ask</b> | Claude (called directly, pay-per-use) reads the free-text email and pulls out structured criteria: sector, stage, geography, check size, deadline, keywords.                                                                 |
| <b>3. Match</b>              | Hard filters (sector/stage/geography must match) narrow the 600+ startup records, then the same kind of deterministic keyword/taxonomy scoring already proven on Task 1 ranks what's left — no external service call needed. |
| <b>4. Shortlist draft</b>    | Top ~15-20 startups, each with a one-line 'why this fits' note from Claude, land in a review screen inside the same admin panel staff already use for the website.                                                           |
| <b>5. Human review</b>       | A team member skims the list, adds or removes entries, then approves. Nothing is sent to a requester without this step.                                                                                                      |
| <b>6. Send + log</b>         | Approved shortlist goes out through the same outbound mailbox already configured for the site; the request, the shortlist, and any edits are logged in the same database for later reference.                                |

This mirrors a pattern already proven on Task 1: a deterministic layer (hard filters, keyword scoring) always decides who is eligible and how they rank, and AI is used only where it adds judgement — reading free text and writing a one-line explanation — never for the final send decision. The startup records and incoming requests live as two new tables in the database the website already uses, reviewed through the same admin login staff already have. This is a deliberate choice over standing up a second application: the server is a modest 4GB/2-CPU box that already needs swap space just to build the existing site, so adding logic to the app in place is safer than running two Node processes side by side. If load ever becomes a real constraint, moving to a slightly larger VPS is a few euros a month, not a re-architecture.

## Tools and estimated running cost

| Item                                          | Tool                                                                                                            | Est. monthly cost              |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------|
| Hosting (app + dashboard)                     | Same Hetzner VPS already running the startin.lv site — new logic added to the existing app, not a second server | €0 marginal (already paid for) |
| Startup + request database                    | Same Postgres instance already on that VPS, two new tables                                                      | €0 marginal                    |
| Inbound email                                 | Existing team mailbox + a small polling script on the same server                                               | €0 marginal                    |
| Outbound email                                | Same SMTP account already configured for the site                                                               | €0 marginal                    |
| Matching logic (filters + scoring)            | Runs inside the existing app, no external call                                                                  | €0                             |
| AI (criteria extraction, shortlist rationale) | Claude API, pay-per-use, ~50 requests/month                                                                     | €10 - €25                      |

|                       |  |                                |
|-----------------------|--|--------------------------------|
| <b>Total new cost</b> |  | <b>roughly €10 - €25/month</b> |
|-----------------------|--|--------------------------------|

Nearly everything reuses infrastructure already paid for. The only genuinely new, unavoidable cost is the AI API usage, since running a comparable model ourselves isn't worth the effort at this volume. That's a small fraction of the ~28 staff-hours/month this replaces, and a tenth of what a managed SaaS pipeline (hosting + database + inbound/outbound email services) would run.

## Keeping the database current and matches accurate

- Self-serve updates: each startup gets a link to review/update its own profile (sector, stage, one-line pitch, contact) — a public page on the same site, submissions landing in a moderation queue rather than going live directly, sent quarterly so the data stays fresh by the people who know it best.
- New startups: the same form, plus a light watch on known local startup lists/events, feeds that same queue for a human to approve before a new entry is used in matching.
- Staleness flag: any profile not confirmed in 6 months is flagged in the admin panel and down-weighted in matching until refreshed.
- Accuracy over time: every time a reviewer adds or removes a startup from a suggested shortlist, that override is logged — recurring corrections point to a filter or keyword that needs adjusting.

## Where a human stays in the loop

- Nothing is emailed to a requester automatically — a person always reviews and clicks send.
- If the AI can't confidently work out what the requester wants, the request is flagged for manual triage instead of guessing.
- Reviewers can add or remove startups before sending; the system never overrides a human choice.
- Periodic spot-checks compare a sample of automated shortlists against what a person would have picked manually, to catch drift early.

## What ships in Week 1 vs. Month 3

|                           | <b>Week 1</b>                                                                                                                           | <b>Month 3</b>                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Database</b>           | Excel imported into a new table in the site's existing database (a few hours of mapping/scripting)                                      | Same database, kept fresh automatically (see below)                                    |
| <b>Matching</b>           | Manual: staff re-types the ask into a filter/search screen in the admin panel, gets a ranked list in seconds instead of scrolling Excel | Automatic: email parsed by AI into criteria, matched, and scored without retyping      |
| <b>Email handling</b>     | Still read and actioned by a person, through the existing mailbox                                                                       | Inbound polling automatic; outbound is a one-click approve & send from the admin panel |
| <b>Data freshness</b>     | Whatever is in the current spreadsheet                                                                                                  | Startups self-update via a public form; stale (6mo+) records flagged                   |
| <b>New infrastructure</b> | None — reuses the site's existing server, database and admin login                                                                      | None — still the same server; only the AI usage is a new bill                          |
| <b>Outcome</b>            | Cuts most of the 7 hrs/week immediately, near-zero build risk                                                                           | Near-zero manual effort per request; quality improves from logged feedback             |

Week 1 alone removes most of the manual searching — the biggest time cost — with almost no build risk. Month 3 removes the remaining manual steps (reading the email, retyping the ask, keeping the sheet current) once the matching rules have been tested against a few weeks of real requests.